

```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

library(table1)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)

```

```

    stop("pH_super approximation relative error exceeds 10%")
    z1_frac <- z1_q
    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria

hr.threshold <- 1.25 # Initital candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

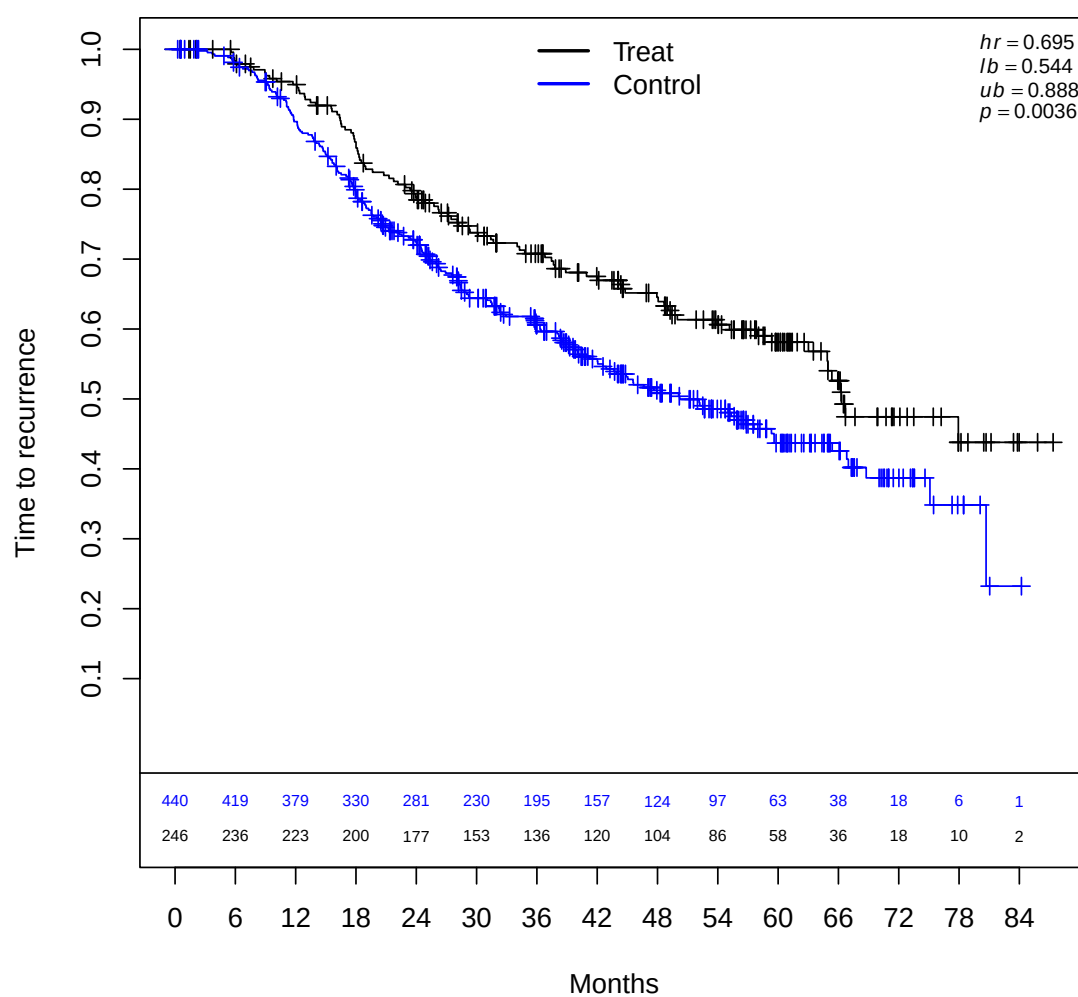
mod.harm <- "alt"
hrH.target <- 2
# out.loc = NULL turns off file creation

```

```

N <- 1000
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)

```



```

## Super-population empirical harm and non-harm hazard ratios= 2.000007 0.6466405
## Causal HR (empirical ITT)= 0.7057463

```

```

file_out <- c("output/sim-FS4_N1k_hrH=2_sim18.Rdata")

# sim sensH ppvH sensHc ppvHc lasso_omit 1: 78 0.4402985 0.5412844 0.9422633
# 0.9158249 1 2: 164 0.6146789 0.5275591 0.9326599 0.9518900 1 3: 221 0.3149606
# 0.5063291 0.9553265 0.9055375 1 4: 257 0.5528455 0.5666667 0.9407070
# 0.9375000 1 5: 273 0.5422535 0.5065789 0.9125874 0.9233491 1

n_add_noise <- 3

sim <- 257

```

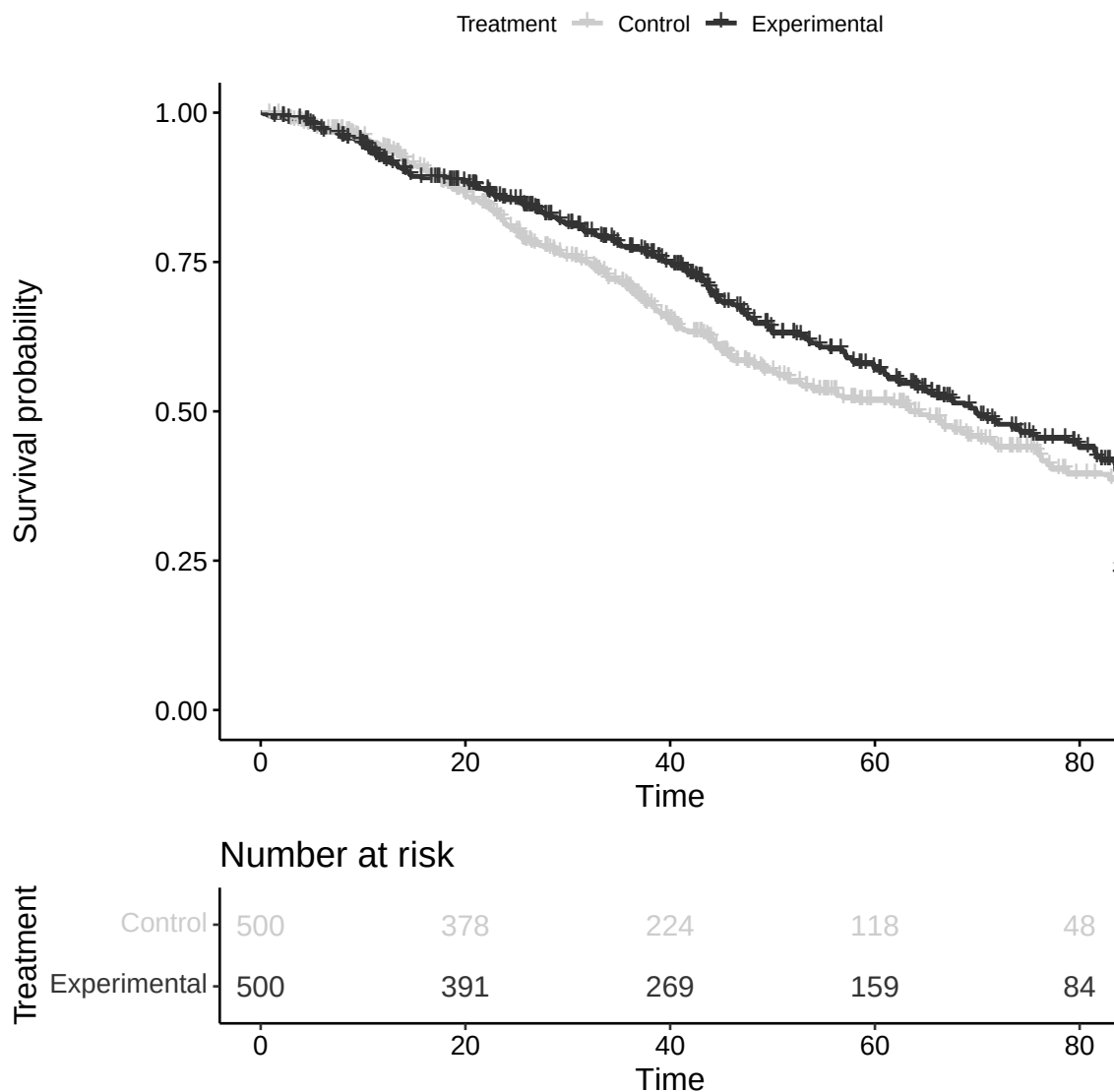
```
# sim <- 18

dgm <- this.dgm$dgm

x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
  legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
  risk.table = TRUE, risk.table.col = "strata")
```



```
if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
```

```

# Add 5 noise
x$noise1 <- rnorm(N)
x$noise2 <- rnorm(N)
x$noise3 <- rnorm(N)
x$noise4 <- rnorm(N)
x$noise5 <- rnorm(N)
confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
  "noise2", "noise3", "noise4", "noise5")
}

if (n_add_noise == 3) {
  set.seed(8316951 + 1000 * sim)
  # Add 3 noise
  x$noise1 <- rnorm(N, sd = 1)
  x$noise2 <- rnorm(N, sd = 1)
  x$noise3 <- rnorm(N, sd = 1)
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
    "noise2", "noise3")
}

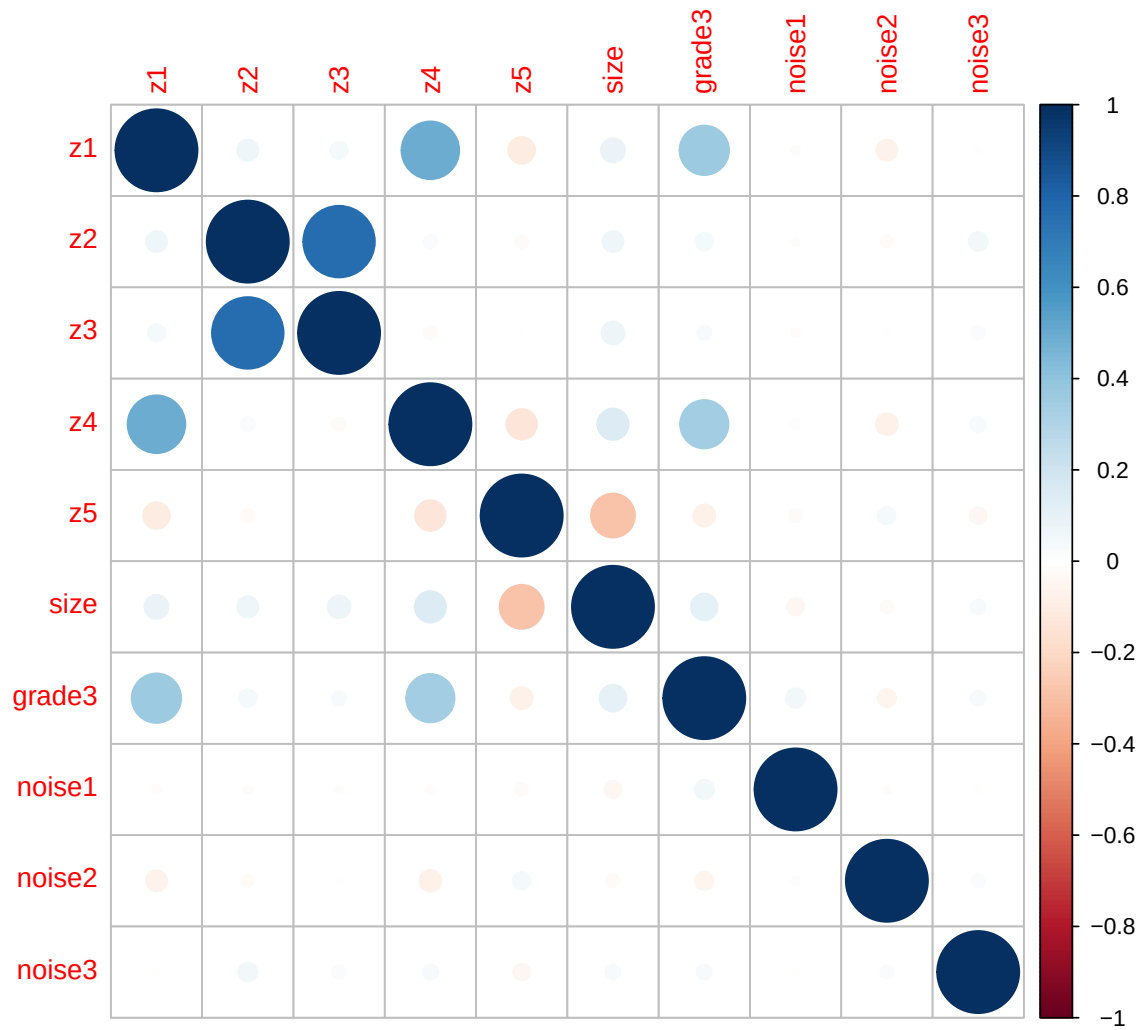
# More challenging which_replace <- which(confounders.name=='z1')
# confounders.name[which_replace] <- 'er'

cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+z1+z2+z3+z4+z5+size+grade3+noise1
coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef) se(coef)      z      p
## treat  -2.802e-01  7.556e-01  8.833e-02 -3.172 0.001513
## z1       3.969e-01  1.487e+00  1.089e-01  3.644 0.000268
## z2      -1.832e-01  8.326e-01  1.355e-01 -1.352 0.176343
## z3       1.064e-01  1.112e+00  1.368e-01  0.778 0.436701
## z4       5.089e-01  1.663e+00  1.049e-01  4.852 1.22e-06
## z5      -1.046e+00  3.513e-01  9.482e-02 -11.034 < 2e-16
## size     8.521e-05  1.000e+00  3.117e-03  0.027 0.978192
## grade3   1.284e-01  1.137e+00  1.014e-01  1.266 0.205644
## noise1  -2.552e-03  9.975e-01  4.365e-02 -0.058 0.953387
## noise2   1.529e-02  1.015e+00  4.494e-02  0.340 0.733765
## noise3  -3.035e-02  9.701e-01  4.260e-02 -0.712 0.476240
##
## Likelihood ratio test=245.4 on 11 df, p=< 2.2e-16
## n= 1000, number of events= 536

Zm <- cor(as.matrix(x[, c(confounders.name)]))
corrplot(Zm)

```



```
suppressWarnings(table1(~z1 + z2 + z3 + z4 + z5 + size + grade3 + grade + noise1 +
  noise2 + noise3 | treat, data = x))
```

	0	1	Overall
	(N=500)	(N=500)	(N=1000)
z1			
Mean (SD)	0.268 (0.443)	0.266 (0.442)	0.267 (0.443)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
z2			
Mean (SD)	0.548 (0.498)	0.532 (0.499)	0.540 (0.499)
Median [Min, Max]	1.00 [0, 1.00]	1.00 [0, 1.00]	1.00 [0, 1.00]
z3			
Mean (SD)	0.432 (0.496)	0.414 (0.493)	0.423 (0.494)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
z4			
Mean (SD)	0.522 (0.500)	0.492 (0.500)	0.507 (0.500)
Median [Min, Max]	1.00 [0, 1.00]	0 [0, 1.00]	1.00 [0, 1.00]
z5			
Mean (SD)	0.520 (0.500)	0.548 (0.498)	0.534 (0.499)
Median [Min, Max]	1.00 [0, 1.00]	1.00 [0, 1.00]	1.00 [0, 1.00]
size			
Mean (SD)	28.2 (13.3)	30.0 (14.4)	29.1 (13.9)
Median [Min, Max]	25.0 [4.00, 100]	27.0 [3.00, 100]	25.0 [3.00, 100]
grade3			
Mean (SD)	0.262 (0.440)	0.274 (0.446)	0.268 (0.443)
Median [Min, Max]	0 [0, 1.00]	0 [0, 1.00]	0 [0, 1.00]
grade			
Mean (SD)	2.15 (0.593)	2.17 (0.589)	2.16 (0.591)
Median [Min, Max]	2.00 [1.00, 3.00]	2.00 [1.00, 3.00]	2.00 [1.00, 3.00]
noise1			
Mean (SD)	0.00753 (0.987)	-0.0719 (0.974)	-0.0322 (0.981)
Median [Min, Max]	-0.0236 [-3.04, 2.80]	-0.0621 [-3.27, 2.48]	-0.0412 [-3.27, 2.80]
noise2			
Mean (SD)	0.0268 (0.951)	0.0744 (1.01)	0.0506 (0.979)
Median [Min, Max]	0.0521 [-3.47, 2.77]	0.0636 [-2.58, 3.11]	0.0603 [-3.47, 3.11]
noise3			
Mean (SD)	-0.0197 (0.944)	-0.0334 (1.05)	-0.0266 (1.00)
Median [Min, Max]	-0.0573 [-3.37, 2.75]	-0.0292 [-4.23, 2.87]	-0.0430 [-4.23, 2.87]

Options Allconfounders.name is list of confounders within analysis dataset
(1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
Lasso used to reduce dimension Continuous covariates are cut at medians
However, if GRF selects a covariate cut then only that cut is used: For
example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
<= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
F & use_grf = T Median cuts (unless selected via GRF) as in (2) However no
possible dimension reduction via lasso All categorical factors included

```
use_lasso <- TRUE
use_grf <- TRUE
use_grf_only <- FALSE
```

```

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 12,
  frac.tau = 1, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
  event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
  hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
  pstop_futile = pstop_futile, pconsistency.threshold = pconsistency.threshold,
  stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
  plot.sg = TRUE, vi.grf.min = vi.grf.min)

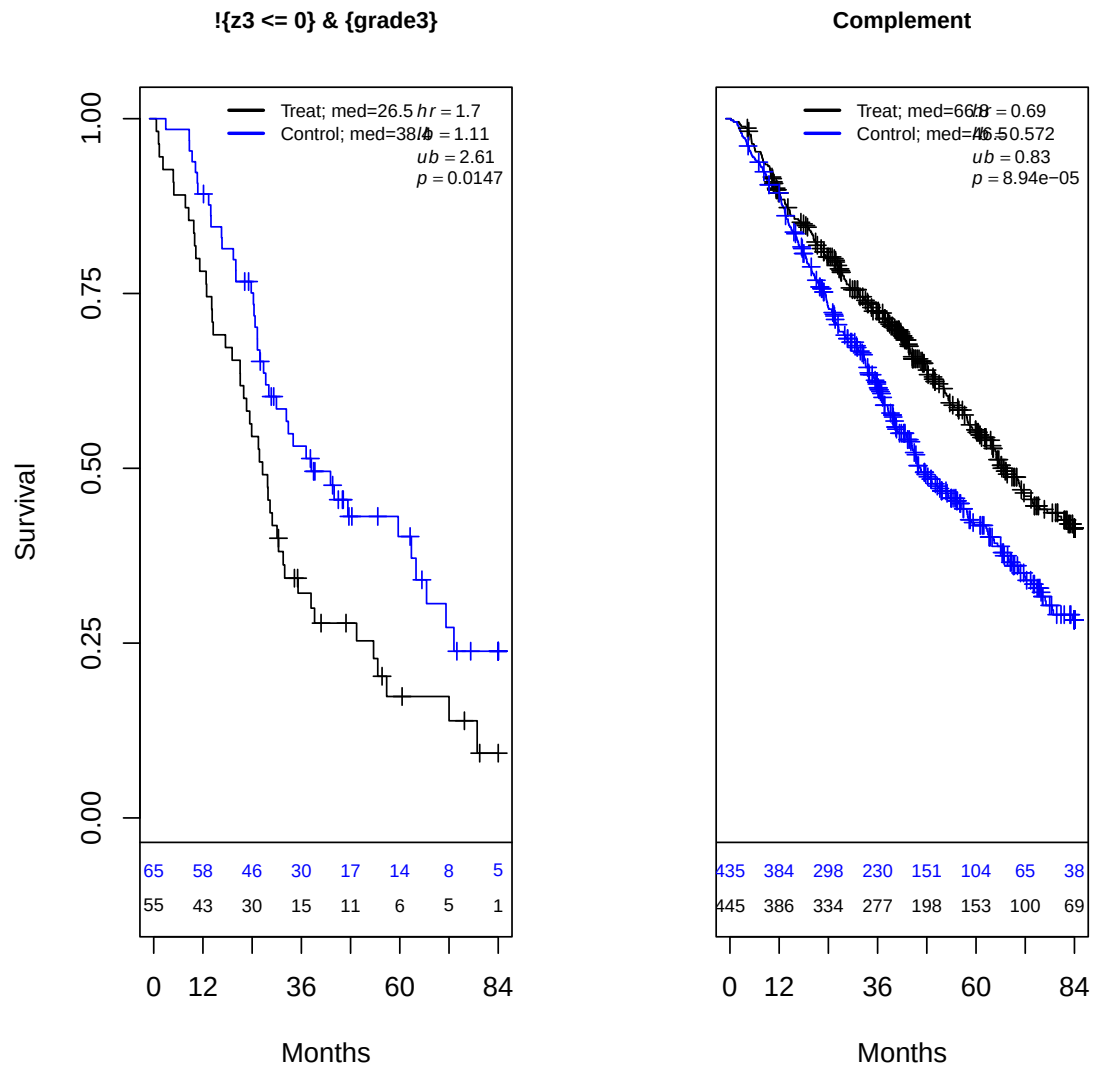
## tau, maxdepth= 83.52584 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2   -5.670317   950.000000   1.765932    5.670317   950.000000
## 2         4    5.952034   108.000000   5.266758   -5.952034   108.000000
## 3         5   -8.970315   624.000000   2.162628    8.970315   624.000000
## 4         6   -8.874432   148.000000   4.350854    8.874432   148.000000
## 5         7   10.784032   120.000000   4.829520   -10.784032   120.000000
##   treated.se      diff Nsg depth
## 1   1.765932 -11.34063 950     1
## 2   5.266758  11.90407 108     2
## 3   2.162628 -17.94063 624     2
## 4   4.350854 -17.74886 148     2
## 5   4.829520  21.56806 120     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 5         7    10.78403    120.00000    4.82952   -10.78403    120.00000
##   treated.se      diff Nsg depth
## 5   4.82952 21.56806 120     2
## [1] "grade3 <= 0"      "noise3 <= -1.05" "z3 <= 0"
## [1] "z3 <= 0"
## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade3
## 10 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## z1      0.376064298
## z2     -0.009245902
## z3      .
## z4      0.476468319
## z5     -0.964676385
## size      .
## grade3  0.065333207
## noise1   .
## noise2   .
## noise3 -0.006719952
## Cox-LASSO selected: z1 z2 z4 z5 grade3 noise3
## Cox-LASSO not selected: z3 size noise1 noise2
## Median cuts (after Lasso if use_lasso) noise3
## Categorical (after Lasso if use_lasso) z1 z2 z4 z5 grade3
## Prior to removing (if in GRF): noise3
## Initial GRF cuts included grade3 <= 0 noise3 <= -1.05 z3 <= 0
## Dropping variables (flag_drop): size grade3
## # of candidate subgroup factors= 7
## [1] "noise3 <= -1.05" "z1"          "z2"          "z4"
## [5] "z5"              "grade3"       "z3 <= 0"
## At least 1 variable droppedConfounders per grf screening q3 q5 q7 q4 q1 q2 q6
##   FSconfounders.name      vi.cs vi_ratio
## 3                   q3 0.1817732 1.2724123

```

```

## 5          q5 0.1731989 1.2123921
## 7          q7 0.1527927 1.0695491
## 4          q4 0.1296114 0.9072799
## 1          q1 0.1244171 0.8709199
## 2          q2 0.1221717 0.8552021
## 6          q6 0.1160349 0.8122445
## Number of unique levels (L) and possible subgroups= 14 16383
## # of subgroups based on # variables > k.max and excluded (per million) 0.016278
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 10 9
## # of subgroups with sample size less than criteria 13
## # of subgroups meeting all criteria = 92
## # of subgroups fitted (Cox model estimable) = 92
## *Subgroup Searching Minutes=* 0.008333333
## Number of subgroups meeting HR threshold 6
## # of candidate subgroups (meeting HR criteria) = 6
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K    n    E d1    m1    m0    HR L(HR) U(HR) q3.0 q3.1 q5.0 q5.1 q7.0 q7.1
## 1:  2 120  85 45 26.54 38.42 1.70  1.11  2.61    0    0    0    0    1    0
## 2:  2 123  97 49 21.07 30.53 1.61  1.08  2.40    0    0    0    0    1    0
## 3:  2 155 102 53 30.07 43.11 1.35  0.91  1.99    0    1    0    0    0    0
## 4:  2  77  32 19 63.74    NA 1.32  0.65  2.68    0    0    0    1    0    0
## 5:  2 159 117 59 28.36 32.82 1.26  0.88  1.82    0    1    0    0    0    0
## 6:  2  71  42 22 43.69 59.61 1.26  0.69  2.32    0    1    0    0    0    0
## 7: NA  NA  NA NA    NA    NA  NA    NA    NA    NA    NA    NA    NA    NA    NA
## 8: NA  NA  NA NA    NA    NA  NA    NA    NA    NA    NA    NA    NA    NA    NA
## 9: NA  NA  NA NA    NA    NA  NA    NA    NA    NA    NA    NA    NA    NA    NA
## 10: NA NA  NA NA    NA    NA  NA    NA    NA    NA    NA    NA    NA    NA    NA
##      q4.0 q4.1 q1.0 q1.1 q2.0 q2.1 q6.0 q6.1
## 1:    0    0    0    0    0    0    0    1
## 2:    0    0    0    0    0    1    0    0
## 3:    0    0    0    0    0    0    0    1
## 4:    0    0    0    1    0    0    0    0
## 5:    0    0    0    0    0    1    0    0
## 6:    0    0    0    1    0    0    0    0
## 7:   NA   NA   NA   NA   NA   NA   NA   NA
## 8:   NA   NA   NA   NA   NA   NA   NA   NA
## 9:   NA   NA   NA   NA   NA   NA   NA   NA
## 10:  NA   NA   NA   NA   NA   NA   NA   NA
## Consistency 0.99
## # of splits= 400
## Model, % Consistency Met= !{z3 <= 0} {grade3} 0.99
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K          M.1          M.2
## 1:          0.99 120          5          1 2 !{z3 <= 0} {grade3}

```



```
## [1] "!{z3 <= 0}" "{grade3}"
## % consistency criteria met= 0.99
## SG focus= hr
## Subgroup Consistency Minutes= 0.02448333
## Subgroup found
## Minutes overall= 0.0623

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, details = TRUE)

## tau, maxdepth= 83.52584 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2    -5.578387   950.000000   1.770696     5.578387     950.000000
## 2         4    -9.606748   488.000000   2.470483     9.606748     488.000000
## 3         5     4.436213    89.000000   6.029551    -4.436213     89.000000
## 4         6    -7.169513   300.000000   3.128223     7.169513     300.000000
## 5         7    12.756450   123.000000   4.335333    -12.756450     123.000000
##   treated.se      diff Nsg depth
```

```
## 1 1.770696 -11.156774 950 1
## 2 2.470483 -19.213496 488 2
## 3 6.029551 8.872427 89 2
## 4 3.128223 -14.339025 300 2
## 5 4.335333 25.512901 123 2
## leaf.node control.mean control.size control.se treated.mean treated.size
## 5 7 12.756450 123.000000 4.335333 -12.756450 123.000000
## treated.se diff Nsg depth
## 5 4.335333 25.5129 123 2
## [1] "z3 <= 0" "noise2 <= 1.05" "z1 <= 0"
## [1] "z1 <= 0"
```

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_")

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")
## Minutes for Boots 1000 4.435417

cat("Projection per 1000", c(tB.min * (1000/NB)), "\n")
## Projection per 1000 4.435417

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")
```

```

## Propn bootstrap subgroups found = 0.953
# How many timed out
cat("Number timed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmins_search > max.minutes)),
    "\n")

## Number timed out= 0
H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)
Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)

print(H_estimates)
##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower H1_upper
## 1: 1.702336 0.3712214 1.110267 2.610137 1.338556 0.3297265 0.8259617 2.169267
print(Hc_estimates)
##           H0          sdH0 H0_lower H0_upper          H1          sdH1 H1_lower
## 1: 0.6895097 0.06543284 0.572484 0.8304574 0.7065864 0.09753191 0.5391031
##           H1_upper
## 1: 0.926102

# H_estimates<-get_dfRes(Hobs=H_obs,seHobs=seH_obs,H1_adj=resB$H_biasadj_1,ystar=Ystar_mat,cov_metho
# Hc_estimates<-get_dfRes(Hobs=Hc_obs,seHobs=seHc_obs,H1_adj=resB$Hc_biasadj_1,ystar=Ystar_mat,cov_m
# print(H_estimates) print(Hc_estimates)

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")
## Overall minutes for analysis 4.5845
save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, cox.formula.boot, file = file_out)

# ITT causal hazard ratio
mean(fs.est$df.est$hlin.ratio)
## [1] 0.8064276

# H Oracle
dfH <- subset(fs.est$df.est, flag.harm == 1)
mean(dfH$hlin.ratio)
## [1] 2.249393
dfH_est <- subset(fs.est$df.est, treat.recommend == 0)
mean(dfH_est$hlin.ratio)
## [1] 1.536411

# Hc Oracle
dfHc <- subset(fs.est$df.est, flag.harm == 0)
mean(dfHc$hlin.ratio)
## [1] 0.6040505
dfHc_est <- subset(fs.est$df.est, treat.recommend == 1)
mean(dfHc_est$hlin.ratio)
## [1] 0.7068844

# dfSens <- subset(fs.est$df.est, flag.harm==1 & treat.recommend==0) sensH <-
# nrow(dfSens)/nrow(dfH)

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT	0.77	0.649	0.913	1000	.
H subgroup estimates					
H_{oracle}	1.611	1.08	2.404	123	.
$\hat{H}$	1.702	1.11	2.609	120	68
$\hat{H}_{bc}$	1.339	0.826	2.171	120	68
H-complement subgroup estimates					
H_{oracle}^c	0.703	0.582	0.85	877	.
$\hat{H}^c$	0.69	0.574	0.83	880	825
$\hat{H}_{bc}^c$	0.707	0.539	0.928	880	825

```
## Confounders evaluated in Forestsearch:
## [1] "noise3 <= -1.05" "z1"           "z2"           "z4"
## [5] "z5"           "grade3"       "z3 <= 0"
```